

PHP

COMPLETE NOTES

Easy Language • Simple Examples • Important Topics Highlighted

★ Red star = Important Topic (read carefully, exam / interview favorite)

MMOC TEACH
MADE BY PRAVENDRA

INDEX

★ 1. Introduction to PHP

- 2. Variables and Data Types
- 3. Operators
- 4. Conditional Statements (if, elseif, else)
- 5. Loops - for, while, foreach

★ 6. Functions

★ 7. Arrays

★ 8. Superglobals - \$_GET and \$_POST

- 9. Form Handling

★ 10. Sessions and Cookies

- 11. Including Files (include, require)

★ 12. Object-Oriented PHP (Classes)

★ 13. Connecting to MySQL Database

- 14. Error Handling (try, catch)

★ 1. Introduction to PHP *[IMPORTANT TOPIC]*

Explanation:

PHP (Hypertext Preprocessor) is a programming language specially made for building websites. Unlike JavaScript, PHP code runs on the SERVER (not the browser), and only the final result (usually HTML) is sent to the user's browser.

PHP is widely used to build dynamic websites — pages that can change based on user input, like login systems, blogs, and online stores.

Key Points:

- Runs on the server, not in the browser.
- Works very well with MySQL databases.
- PHP files use the .php extension.
- PHP code is written inside `<?php ... ?>` tags.

Example Code:

```
<?php
    echo "Hello, World!";
?>
```

Output:

Hello, World!

💡 *Note: echo is used to print output onto the webpage in PHP.*

2. Variables and Data Types

Explanation:

In PHP, every variable name starts with a \$ sign. PHP automatically understands the data type based on the value given — we don't need to declare it separately.

Example Code:

```
<?php
    $name = "Pravendra";
    $age = 21;
    $height = 5.9;

    echo $name . " is " . $age . " years old";
?>
```

Output:

Pravendra is 21 years old

 *Note: The dot (.) symbol is used to join (concatenate) text and variables together in PHP.*

3. Operators

Explanation:

Operators are symbols used to perform actions on values, like maths or comparisons.

Example Code:

```
<?php
    $a = 10;
    $b = 3;
    echo $a + $b;    // 13
    echo "\n";
    echo $a % $b;    // 1
?>
```

Output:

```
13
1
```

4. Conditional Statements (if, elseif, else)

Explanation:

Conditional statements let the program make decisions based on whether something is true or false.

Example Code:

```
<?php
    $marks = 75;

    if ($marks >= 90) {
        echo "Grade A";
    } elseif ($marks >= 60) {
        echo "Grade B";
    } else {
        echo "Grade C";
    }
?>
```

Output:

Grade B

5. Loops - for, while, foreach

Explanation:

Loops repeat a block of code multiple times. 'foreach' is a special loop in PHP made especially for going through each item in an array.

Example Code:

```
<?php
    for ($i = 1; $i <= 3; $i++) {
        echo "Count: $i\n";
    }
?>
```

Output:

```
Count: 1
Count: 2
Count: 3
```

★ 6. Functions *[IMPORTANT TOPIC]*

Explanation:

A function is a reusable block of code that performs a specific task, defined using the 'function' keyword.

Example Code:

```
<?php
    function addNumbers($a, $b) {
        return $a + $b;
    }

    echo addNumbers(5, 7);
?>
```

Output:
12

★ 7. Arrays [IMPORTANT TOPIC]

Explanation:

An array stores multiple values together in one variable. PHP also supports 'associative arrays', where each value has a custom named key instead of just a number.

Example Code:

```
<?php
    $fruits = ["apple", "banana", "mango"];
    echo $fruits[0];

    $student = ["name" => "Pravendra", "age" => 21];
    echo $student["name"];
?>
```

Output:

```
apple
Pravendra
```

★ 8. Superglobals - \$_GET and \$_POST [IMPORTANT TOPIC]

Explanation:

Superglobals are special built-in PHP variables available everywhere in a script. \$_GET collects data sent through the URL (like a search query). \$_POST collects data sent through an HTML form, which is safer for sensitive information like passwords.

Example Code:

```
<?php
// From a URL like: page.php?name=Pravendra
$name = $_GET['name'];
echo "Hello, " . $name;
?>
```

Output:

Hello, Pravendra

 *Note: Never use \$_GET for passwords — the value is visible directly in the URL.*

9. Form Handling

Explanation:

PHP is commonly used to process data submitted from an HTML form, like a login or a contact form, using `$_POST` or `$_GET`.

Example Code:

```
<!-- HTML form -->
<form method="post" action="process.php">
  <input type="text" name="username">
  <input type="submit">
</form>

<?php
  // process.php
  $username = $_POST['username'];
  echo "Welcome, " . $username;
?>
```

Output:

Welcome, (whatever the user typed in the form)

★ 10. Sessions and Cookies *[IMPORTANT TOPIC]*

Explanation:

A Session stores information about a user (like being logged in) on the SERVER, for as long as they use the website. A Cookie stores small pieces of information directly in the user's BROWSER, even after they close the site.

Example Code:

```
<?php
    session_start();
    $_SESSION['username'] = "Pravendra";

    echo $_SESSION['username'];
?>
```

Output:
Pravendra

 *Note: session_start() must be called at the very top of a PHP page, before any output, to use sessions.*

11. Including Files (include, require)

Explanation:

'include' and 'require' let us insert the content of one PHP file into another, useful for reusing common code like a header or footer on every page.

Example Code:

```
<?php
    include 'header.php';
    echo "This is the main page content.";
    include 'footer.php';
?>
```

Output:

(Shows header.php's content, then the main text, then footer.php's content.)

 *Note: 'require' stops the whole script with an error if the file is missing; 'include' only shows a warning and continues.*

★ 12. Object-Oriented PHP (Classes) *[IMPORTANT TOPIC]*

Explanation:

PHP also supports Object-Oriented Programming. A class is a blueprint for creating objects, which group related data and functions together.

Example Code:

```
<?php
class Student {
    public $name;

    function __construct($name) {
        $this->name = $name;
    }

    function showName() {
        echo "Name: " . $this->name;
    }
}

$s1 = new Student("Pravendra");
$s1->showName();
?>
```

Output:

Name: Pravendra

 *Note: In PHP, -> is used to access an object's properties and methods, instead of a dot (.).*

★ 13. Connecting to MySQL Database *[IMPORTANT TOPIC]*

Explanation:

PHP is very commonly used together with a MySQL database to store and retrieve website data, like user accounts or product listings.

Example Code:

```
<?php
    $conn = new mysqli("localhost", "root", "", "mydatabase");

    if ($conn->connect_error) {
        die("Connection failed: " . $conn->connect_error);
    }
    echo "Connected successfully";
?>
```

Output:

Connected successfully

14. Error Handling (try, catch)

Explanation:

Error handling lets us safely catch an unexpected error while the program is running, instead of the whole script crashing.

Example Code:

```
<?php
    try {
        $a = 10;
        $b = 0;
        if ($b == 0) {
            throw new Exception("Cannot divide by zero!");
        }
        echo $a / $b;
    } catch (Exception $e) {
        echo "Error: " . $e->getMessage();
    }
?>
```

Output:

Error: Cannot divide by zero!